pi-monitor-light

A power-budgeted, SSH-only UART monitor and STM32 flashing station

Patrik Drazic
Technical Note

April 2026

Abstract

pi-monitor-light is a single-board headless tool for embedded firmware development: it logs up to four UART streams continuously, flashes STM32 firmware over ST-Link, and is reachable from anywhere on Earth over an SSH-only workflow. The implementation deliberately uses only shell scripts, `systemd`, and `udev` — no Python, no web framework — so that idle CPU is essentially zero and the entire station fits within a 2.3 W power envelope on a Raspberry Pi Zero 2 W. This note documents the motivation, hardware topology, software architecture, power budget, and design decisions behind a deliberate *build down* from a heavier prototype.

Contents

1. Motivation

A prior version of this tool, `pi-monitor`, was a FastAPI web application running on a Raspberry Pi 4 that exposed a browser UI for dual-pane UART monitoring and OpenOCD flashing. It worked, but two constraints made it the wrong shape for the actual deployment.

Power. The target deployment shares a 24 V / 0.2 A rail (4.8 W total) with the rest of a custom carrier PCB. A Pi 4–class board idling at ∼3 W consumes most of that budget before any peripherals are attached. A Pi Zero 2 W idles at well under 1 W, leaving room for the ST-Link and the FTDIs.

Workflow. The browser UI duplicated functionality that is already cleaner over SSH: `tmux` for pane management, `scp` for firmware transfer, `journalctl -f` for live logs. The Python / FastAPI / SSE stack added a permanent ∼50 MB resident footprint and a userspace polling loop per pane to deliver a UI that, in practice, was no longer needed.

The redesign target therefore was: same capabilities; total board power ≤ 2.3 W on a Pi Zero 2 W; no userspace polling; reachable from anywhere on Earth without a public IP.

2. System overview

Figure 1 shows the deployed station. The operator's laptop joins the same Tailscale tailnet as the Pi, then SSHs to it by its tailnet hostname; the Pi never needs an inbound public IP. On the Pi, four small CLI wrappers (`sl-monitor`, `sl-attach`, `sl-flash`, `sl-status`) drive a `systemd` template unit per UART, with USB hot-plug events bridged into systemd by a `udev` rule. The data path itself is a kernel `cat` on the tty device, piped through `ts` for timestamping and `tee` for journaling — no userspace polling loop.

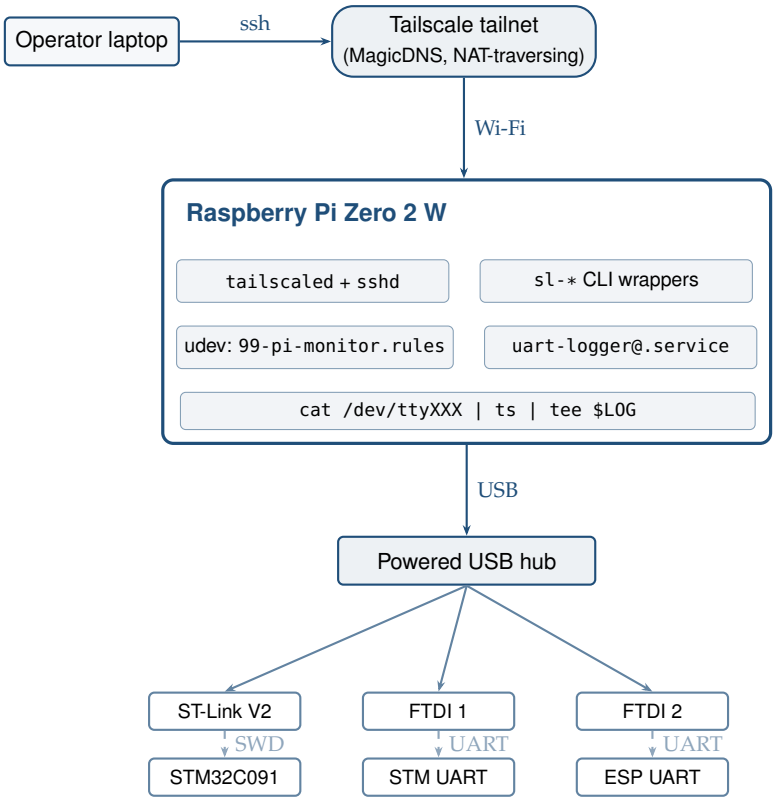

Figure 1. End-to-end topology. Solid lines are TCP/USB; dashed lines are the SWD/UART pin connections to the device under test. The Pi has no public IP; remote reachability comes from Tailscale.

3. Hardware

The station is a small daisy-chain of off-the-shelf parts. The custom carrier PCB supplies 5 V to the Pi via the GPIO header; no SD-card swap, HDMI output, or display is used in production.

4. Software architecture

4.1 Data path: zero userspace polling

The core observation is that "log a UART to disk" is not a problem that needs userspace logic — the kernel already has the relevant primitives. The whole data path is one shell pipeline (Listing 2, line 6): `cat` blocks in the kernel on a serial `read()` until bytes arrive (zero CPU when idle), `ts` from `moreutils` prepends a per-line timestamp, and `tee -a` writes to the log

Component	Role	Idle draw
Raspberry Pi Zero 2 W	SoC + Wi-Fi	0.5–0.7 W
ST-Link V2	STM32 SWD programmer	0.10–0.15 W
FTDI TTL-232R-3V3 $\times N$ (N=1..4)	Per-port UART tap	$\sim$0.05–0.08 W each
Powered USB hub	ST-Link + FTDI fan-out	0.10–0.20 W
24 V $\rightarrow$ 5 V buck	Carrier-PCB power conditioning	—

Table 1. Bill of materials. The Pi Zero 2 W's single micro-USB-OTG port forces the use of a powered hub; this is the only "required" peripheral beyond the targets themselves.

file while forwarding to `systemd`'s journal. There is no userspace polling loop anywhere in the data path.

4.2 Lifecycle: udev + systemd handshake

The non-trivial part is making this resilient across USB hot-plug and across reboots. The mechanism is a two-sided handshake between `udev` and a `systemd` template unit (`uart-logger@.service`):

- **Start** on USB `add`: a udev rule (Listing 1) tags the new `ttyUSB*` or `ttyACM*` device with `TAG+="systemd"` and `ENV{SYSTEMD_WANTS}+="uart-logger@%k.service"`, which causes `systemd` to instantiate the per-device unit.
- **Stop** on USB `remove`: `BindsTo=dev-%i.device` in the unit makes `systemd` follow the device unit's lifecycle. Removal stops the logger and triggers `ExecStopPost` to write a session-end marker plus duration.
- **Restart** on transient failure: `Restart=on-failure` handles the case where the device is still present but the pipeline died (e.g. EIO). Importantly, this does *not* fire on `BindsTo`-driven stops — those are systemd-driven and are explicitly excluded by `Restart=` [1]. The udev re-fire on the next `add` is the recovery path.

Listing 1. `udev` rule that bridges USB hot-plug into systemd. Both `ttyUSB*` (FTDI, CP210x) and `ttyACM*` (CDC-ACM) are matched.

```
SUBSYSTEM=="tty", KERNEL=="ttyUSB[0-9]*", ACTION=="add", \
  TAG+="systemd", ENV{SYSTEMD_WANTS}+="uart-logger@%k.service"

SUBSYSTEM=="tty", KERNEL=="ttyACM[0-9]*", ACTION=="add", \
  TAG+="systemd", ENV{SYSTEMD_WANTS}+="uart-logger@%k.service"
```

The unit (Listing 2) runs as a dedicated system user `pi-monitor` and is sandboxed by `ProtectSystem=strict`, `ProtectHome=yes`, `NoNewPrivileges=yes`, with `ReadWritePaths=` restricted to the log and runtime directories.

Listing 2. Excerpt of the systemd template unit. The `ExecStart` pipeline is the entire data path; everything else is policy. `ConditionPathExists` silently skips the unit for ports not present in `ports.conf`, avoiding "failed unit" clutter for irrelevant USB-serial devices.

```
[Unit]
Description=UART logger on /dev/%i
After=dev-%i.device
BindsTo=dev-%i.device
ConditionPathExists=/etc/pi-monitor-light/ports.conf.env-%i

[Service]
User=pi-monitor
```

```
EnvironmentFile=/etc/pi-monitor-light/ports.conf.env-%i
ExecStartPre=/usr/bin/stty -F /dev/%i ${BAUD} raw -echo -crtscts
ExecStart=/bin/sh -ec 'set -o pipefail; \
  LOG="/var/log/pi-monitor/${NAME}/$(date +%%Y-%%m-%%d_%%H-%%M-%%S).log"; \
  exec /usr/bin/cat /dev/%i | /usr/bin/ts | /usr/bin/tee -a "$LOG"'
Restart=on-failure
ProtectSystem=strict
ProtectHome=yes
NoNewPrivileges=yes
```

4.3 Operator interface

Five thin shell wrappers expose the system to the operator. `sl-monitor up | down | restart` enables or disables logger units based on `ports.conf`; `sl-attach` opens a `tmux` session with one window per active port running `journalctl -u uart-logger@<port> -f`; `sl-flash <bin>` flashes a firmware binary via OpenOCD, rejecting path-traversal, non-`.bin` files, and filenames with shell-special characters that could escape the OpenOCD Tcl `program` command. The `ports.conf` parser enforces device-name format, name charset (`[A-Za-z0-9_-]+`, used as a log subdirectory name), duplicate detection, and a trailing-comment carve-out, so the systemd template instantiation cannot be coerced into a malformed unit name.

4.4 Flashing toolchain

OpenOCD is the only cross-platform open-source SWD programmer that supports the STM32C0 series. The relevant target configuration `tcl/target/stm32c0x.cfg` was added to OpenOCD master *after* the 0.12.0 release; consequently the Bookworm package [3] cannot flash the STM32C091 used here. The installer therefore builds OpenOCD from the project's GitHub mirror [4] during first-time setup. Build time on a Pi Zero 2 W is approximately 30 minutes; an environment override `OPENOCD_JOBS=-j1` exists for the OOM case on the 512 MB device.

5. Power budget

The 2.3 W ceiling derives from the carrier PCB's 24 V / 0.2 A rail (4.8 W total) less the rest of the carrier's load. Table 2 lists the design targets.

Element	Estimated draw
Pi Zero 2 W idle (BT off, ACT LED off, 2 of 4 cores enabled)	0.5–0.7 W
ST-Link V2 idle	0.10–0.15 W
FTDI TTL-232R-3V3 ×2	∼0.10–0.15 W
Powered USB hub overhead	0.10–0.20 W
Total idle (target)	**∼1.0–1.4 W**
Total during a flash (transient peak)	**∼1.8–2.2 W**

Table 2. Power budget. Numbers are design targets; the authoritative test is a USB power meter on the Pi's 5 V rail with all peripherals attached and a flash in progress.

The CPU-side savings come from three layered tweaks applied via `config.txt` and `cmdline.txt` [2]: **(1)** `dtoverlay=disable-bt` disables the Bluetooth controller; **(2)** `maxcpus=2` parks two of the four Cortex-A53 cores — idle is nearly identical to single-core, and flash is faster than `maxcpus=1` without the OOM risk during the OpenOCD build; **(3)** ACT LED off and boot splash disabled. Headroom against the 2.3 W ceiling is therefore on the order of 100–500 mW depending on flash load — thin enough that it has to be measured rather than computed. A

documented fallback (`maxcpus=1`, `arm_freq=600`) exists for the case where measured draw exceeds budget.

6. Operator workflow

Once installed (`sudo ./install.sh` on a fresh Bookworm Lite image), the daily workflow from any device on the tailnet is:

```
ssh patrik@pi-monitor
sl-attach                                   # tmux, one window per UART
scp firmware.bin pi-monitor:/var/lib/pi-monitor/firmware/
sl-flash /var/lib/pi-monitor/firmware/firmware.bin
```

Per-site Wi-Fi onboarding uses a phone hotspot with a fixed SSID as a stable bootstrap: the Pi always knows that hotspot, joins it whenever in range, and from that initial connection the operator adds the local Wi-Fi via `nmcli device wifi connect`. The site credentials persist across reboots and the phone hotspot is no longer needed once the site is known. The `README.md` in the repository contains the operator-facing reference for these flows.

7. Design discussion

A handful of choices visibly shaped the final system.

Shell + systemd over a daemon. A Go or Rust daemon would be roughly the same lines of code but introduces a long-running process to maintain. `systemd` already solves unit lifecycle, restart policy, sandboxing, log rotation, and template instantiation. Implementing those once in shell glue and delegating the rest avoided a class of recurring engineering work. The 47-test `bats` suite that came out of this is genuinely small — the parser library is the only piece with non-trivial logic.

Tailscale over SSH-on-public-IP. Tailscale gives transitive zero-config NAT traversal and built-in SSH authentication via the tailnet identity [5], removing port forwarding, dynamic DNS, and per-host SSH key distribution. The operational overhead on the Pi is a single `tailscaled` process (∼15 MB resident, negligible idle CPU). For a tool that travels to client sites with arbitrary Wi-Fi, this is the difference between "works everywhere" and "works on the LAN where you set it up."

New log file per reconnect, not one continuous file. The previous `pi-monitor` version maintained one file per UI session and inlined `-- disconnected -- / -- connected --` markers. The current design produces one file per `add`-to-`remove` cycle of the device. The data is the same; the difference is whose responsibility it is. Letting systemd plus the filesystem be the system of record (one file per unit start) is simpler than threading custom backoff state through a userspace process.

Source-built OpenOCD with a contrib udev rule. A subtlety caught during review: `make install` for OpenOCD does *not* drop its `contrib/60-openocd.rules` into `/etc/udev/rules.d/`, and without it the operator user gets `LIBUSB_ERROR_ACCESS` on the ST-Link USB endpoint. The installer copies the rule explicitly and adds the operator to `plugdev`. This is exactly the kind of cross-task integration issue that per-component tests cannot catch.

8. Limitations and future work

The design intentionally excludes a browser UI, live preview before session start, command injection back to the device, and off-box log aggregation. None of those have been needed in practice; if they become so, the natural place to add them is a separate companion service that consumes the existing log files rather than reshaping the data path.

The 2.3 W ceiling has been computed but not measured. The authoritative test is a USB power meter on the Pi's 5 V rail with all peripherals attached and a flash in progress; if measured draw exceeds 460 mA, the documented fallback path is single-core operation (`maxcpus=1`) and a frequency cap (`arm_freq=600`), which the design analysis suggests would buy another ∼200–300 mW at modest cost to flash time.

The OpenOCD master tracking is convenient now but accrues maintenance debt: when upstream cuts a 0.13.x release with stable STM32C0 support, the installer should switch to the apt-installed package and delete the from-source build path. The skip-check regex already accommodates the version transition.

References

[1] systemd Project. *systemd.service(5) man page — Restart=*. https://www.freedesktop.org/software/systemd/man/latest/systemd.service.html#Restart=

[2] Raspberry Pi Ltd. *Raspberry Pi Documentation — config.txt*. https://www.raspberrypi.com/documentation/computers/config_txt.html

[3] Debian Project. *Package: openocd (0.12.0-1) — Bookworm*. https://packages.debian.org/bookworm/openocd

[4] OpenOCD Project. *openocd-org/openocd — official GitHub mirror*. https://github.com/openocd-org/openocd

[5] Tailscale Inc. *Tailscale SSH*. https://tailscale.com/kb/1193/tailscale-ssh

[6] Raspberry Pi Ltd. *Raspberry Pi Zero 2 W product brief*. https://datasheets.raspberrypi.com/rpizero2/raspberry-pi-zero-2-w-product-brief.pdf